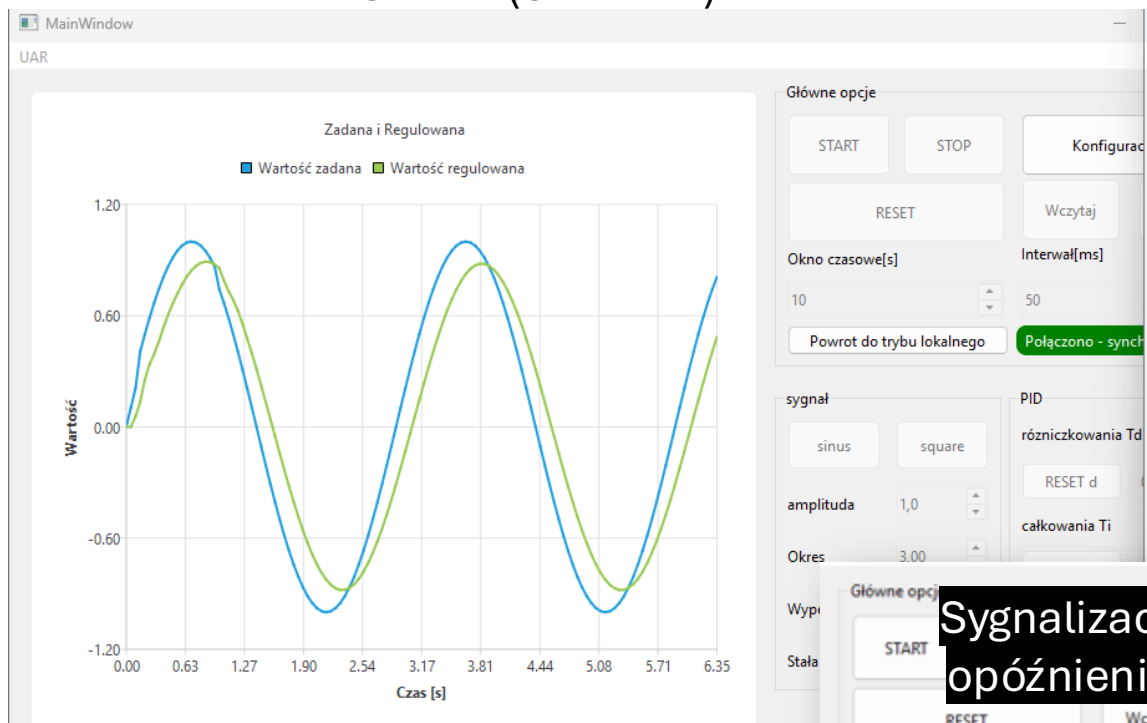


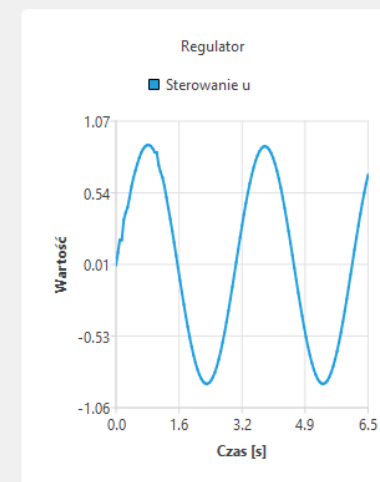
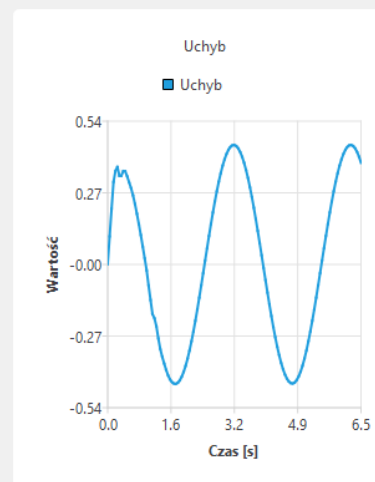
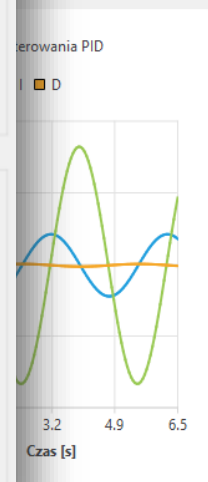
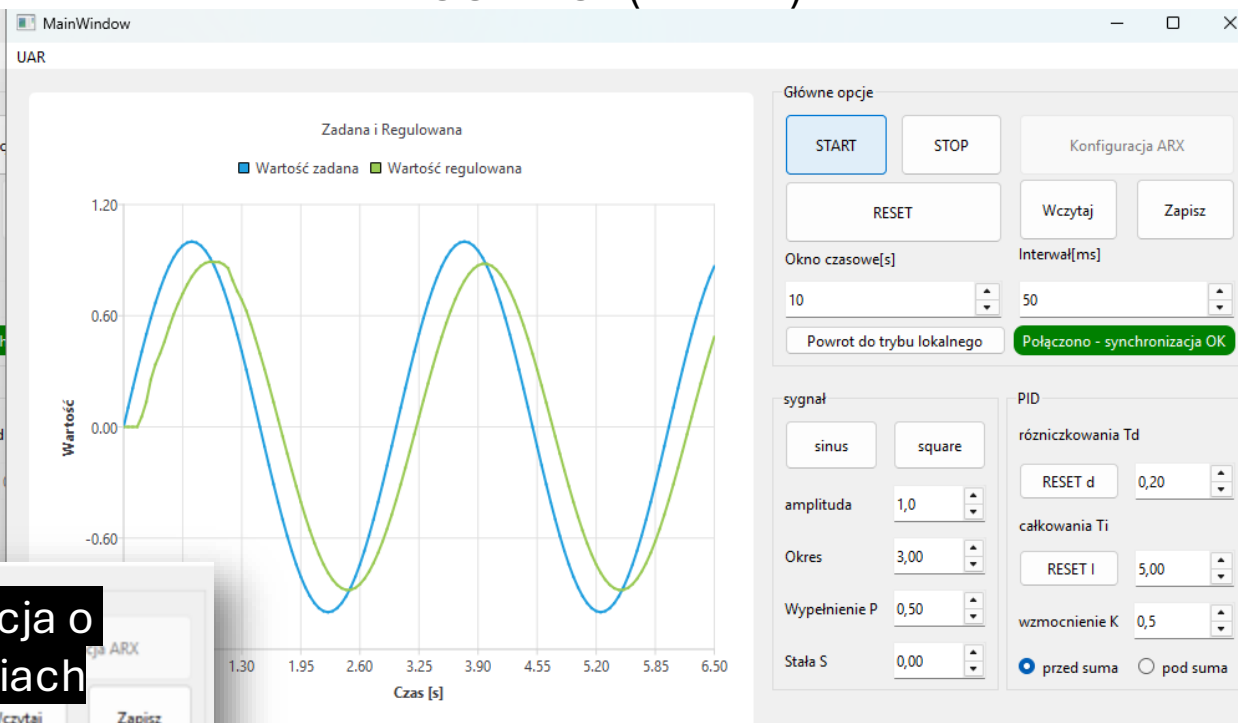
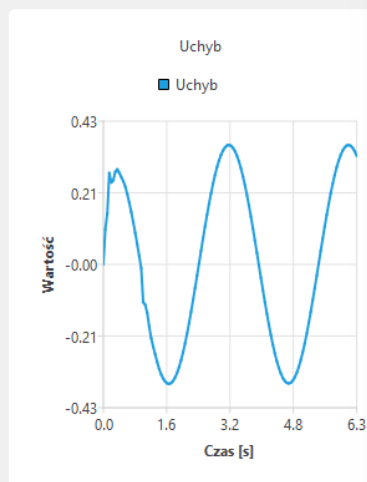
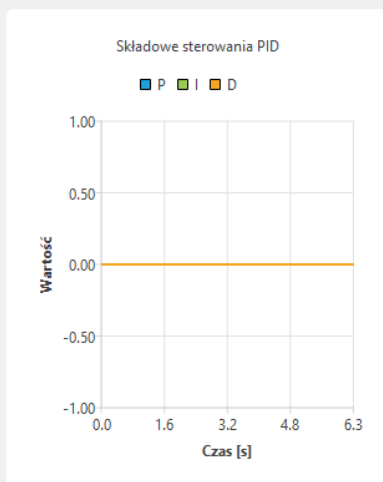
Okres (3) – (4) Krytyczne funkcjonalności sieciowe

- omówić w prezentacji sposoby wykrywania opóźnień w komunikacji oraz sposoby radzenia sobie z nimi. [1.0 pkt]

OBIEKT(SERWER)



Sygnalizacja o opóźnieniach



```

//TRYB REGULATORA SIECIOWEGO

if (oczekiwanieNaY)
{
    ostatniPakietNaCzas = false;
    liczbaSpozniionychPakietow++;
}
else
{
    ostatniPakietNaCzas = true;
}

// jeśli poprzednie y jeszcze nie przyszło pracujemy dalej na ostatnim dostępnym
y = ostatnieYsieciowe;

w = generator.generuj(k);

// uchyb
e = w - y;

// NOWY PID TYLKO GDY PRZYSZLA ODPOWIEDZ
if (!oczekiwanieNaY)
{
    u = pid.symuluj(e);

    oczekiwanieNaY = true;

    emit wyslijSterowanie(u, w);
}

double wartP = getP();
double wartI = getI();
double wartD = getD();

oczekiwanieNaY = true;

emit krokWykonany(w, y, e, u, k, wartP, wartI, wartD);

k++;
}

```

```

void MainWindow::aktualizujStatusSieci()
{
    // brak trybu sieciowego
    if(!client && !server)
    {
        ui->StatusPolaczenia_Label->hide();
        return;
    }

    ui->StatusPolaczenia_Label->show();

    bool polaczono = false;

    // SERVER
    if(server && server->getSocket())
    {
        polaczono = (server->getSocket()->state() == QAbstractSocket::ConnectedState);
    }

    // CLIENT
    if(client)
    {
        polaczono = (client->getSocket()->state() == QAbstractSocket::ConnectedState);
    }

    // BRAK POŁĄCZENIA
    if(!polaczono) {...}

    // OPÓŹNIENIA
    if(!symulator.czyPakietyNaCzas()) {...}

    // OK
    ui->StatusPolaczenia_Label->setText("Połączono - synchronizacja OK");
    ui->StatusPolaczenia_Label->setStyleSheet(
        "background-color: green;"
        "color: white;"
        "border-radius: 8px;"
        "padding: 4px;"
    );
}

```

- omówić działanie końcowej wersji protokołu komunikacyjnego oraz wykorzystany sposób serializacji danych. [1.0 pkt.]

```

// SERIALIZACJA
inline QDataStream& operator<<(QDataStream& out, const ConfigPacket& c)
{
    out << c.Kp << c.Ti << c.Td << c.typCalki

        << c.uMin << c.uMax

        << c.A << c.TRZ << c.P << c.S << c.TT << c.trybGenerators

        << QVector<double>(c.arxA.begin(), c.arxA.end())
        << QVector<double>(c.arxB.begin(), c.arxB.end())

        << c.opoznienie << c.szum

        << c.ograniczenia

        << c.yMin << c.yMax;

    return out;
}

// DESERIALIZACJA
inline QDataStream& operator>>(QDataStream& in, ConfigPacket& c)
{
    QVector<double> a, b;

    in >> c.Kp >> c.Ti >> c.Td >> c.typCalki

        >> c.uMin >> c.uMax

        >> c.A >> c.TRZ >> c.P >> c.S >> c.TT >> c.trybGenerators

        >> a
        >> b

        >> c.opoznienie >> c.szum

        >> c.ograniczenia

        >> c.yMin >> c.yMax;

    c.arxA = std::vector<double>(a.begin(), a.end());
    c.arxB = std::vector<double>(b.begin(), b.end());

    return in;
}

```

```

#pragma once
#include <QDataStream>

struct ControlPacket
{
    double u;
    double w;
};

inline QDataStream& operator<<(QDataStream& out, const ControlPacket& p)
{
    out << p.u << p.w;
    return out;
}

inline QDataStream& operator>>(QDataStream& in, ControlPacket& p)
{
    in >> p.u >> p.w;
    return in;
}

```

```

#pragma once
#include <QDataStream>

struct OutputPacket
{
    double y;
};

inline QDataStream& operator<<(QDataStream& out, const OutputPacket& p)
{
    out << p.y;
    return out;
}

inline QDataStream& operator>>(QDataStream& in, OutputPacket& p)
{
    in >> p.y;
    return in;
}

```

DESERIALIZACJA

```
void MainWindow::on_SpinBox_Stala_editingFinished()
{
    symulator.setGeneratorS(ui->SpinBox_Stala->value());
    if(Tryb == regulator && client)
    {
        wyslijConfigPacket();
    }
}
```

```
void MainWindow::wyslijConfigPacket()
{
    ConfigPacket c = makeConfigPacket(
        client->sendConfig(c);
}
```



SERIALIZACJA

```
void Client::sendConfig(const ConfigPacket& c)
{
    QByteArray buf;
    QDataStream out(&buf, QIODevice::WriteOnly);

    out << quint32(0); // placeholder
    out << quint16(2); // typ = ConfigPacket
    out << c;

    out.device()->seek(0);
    out << quint32(buf.size() - sizeof(quint32));

    socket.write(buf);
}
```

```
void Server::onReadyRead()
{
    buffer.append(socket->readAll());

    while (true)
    {
        if (buffer.size() < 4)
            return;

        QDataStream sizeStream(buffer);
        quint32 size;
        sizeStream >> size;

        if (buffer.size() < 4 + size)
            return;

        QByteArray packet = buffer.mid(4, size);
        buffer.remove(0, 4 + size);

        QDataStream in(packet);
        quint16 type;
        in >> type;

        if (type == 2) {
            ConfigPacket c;
            in >> c;
            emit configReceived(c);
        }
        if (type == 3) // IDK
        {
            ControlPacket p;
            in >> p;

            qDebug() << "SERWER: odebrano u =" << p.u;

            emit sterowanieReceived(p.u, p.w);
        }
    }
}
```



W main window

```
connect(server, &Server::configReceived,
        this, &MainWindow::onConfigPacketReceivedServer);
```

```
void MainWindow::onConfigPacketReceivedServer(const ConfigPacket& c)
{
    symulator.setPID_Kp(c.Kp);
    symulator.setPID_Ti(c.Ti);
    symulator.setPID_Td(c.Td);

    symulator.setGeneratorA(c.A);
    symulator.setGeneratorTRZ(c.TRZ);
    symulator.setGeneratorP(c.P);
    symulator.setGeneratorS(c.S);
    symulator.setGeneratorTT(c.TT);

    symulator.setARX(c.arxA, c.arxB, c.opoznienie, c.szum);
    symulator.setARX_Ograniczenia(c.ograniczenia);
    symulator.setPID_Ograniczenia(c.ograniczenia);
}
```